Université Ibn Khaldoun de Tiaret

Département Informatique

TP N° 2

Git et GitHub : collaboration, fusion et sécurité

Module : DevOps
Enseignant : Abdelkader Ouared

Objectifs pédagogiques

À la fin de ce TP, l'étudiant doit être capable de :

- créer et cloner un dépôt GitHub ;
- utiliser les commandes `add`, `commit`, `push` et `pull` ;
- travailler avec des branches et fusionner des modifications ;
- provoquer, comprendre et résoudre un conflit de fusion ;
- configurer les notifications GitHub ;
- configurer l'authentification SSH avec une paire de clés cryptographiques ;
- utiliser Git indifféremment en ligne de commande ou avec une interface graphique.

1. Contexte et organisation du travail

Vous devez développer en équipe une petite application C de sécurité informatique (par exemple : vérification de la robustesse d'un mot de passe, calcul d'empreintes ou chiffrement simple de messages). Le travail est réalisé dans un dépôt GitHub partagé par trois étudiants. Chaque étudiant contribue à un module différent, puis l'équipe simule une modification concurrente du même fichier afin d'étudier les conflits.

1.1. Répartition des rôles

Étudiant	Responsabilité
Étudiant A	Propriétaire du dépôt ; saisie et validation des entrées : `module1.c`
Étudiant B	Traitement de sécurité (analyse, hachage ou chiffrement) : `module2.c`
Étudiant C	Affichage des résultats et rapport de sécurité : `module3.c`

Le dépôt doit avoir l'organisation suivante :

```
TP-Git-Collaboration/
|-- README.md
|-- main.c
|-- module1.c
|-- module2.c
|-- module3.c
‘-- .gitignore
```

1.2. Choix de l'outil : ligne de commande ou interface graphique

Chaque étudiant peut réaliser le TP au choix :

- en **ligne de commande** (terminal sous Linux, Git Bash sous Windows) ;

— avec un **client graphique** : **GitHub Desktop** ou **TortoiseGit** (Windows, nécessite Git for Windows). D'autres clients équivalents sont acceptés.

Les membres d'une même équipe peuvent utiliser des outils différents : ils travaillent sur le même dépôt. Les commandes données dans l'énoncé sont la référence ; le tableau suivant indique leurs équivalents graphiques. Les intitulés exacts des menus varient selon la version et la langue de l'outil.

Opération	**Ligne de commande**	**GitHub Desktop**	**TortoiseGit (clic droit)**
Cloner	`git clone URL`	`File > Clone repository`	`Git Clone...`
Voir l'état	`git status`	Onglet `Changes`	`Check for modifications`
Voir les différences	`git diff`	Panneau de droite (`Changes`)	`Diff`
Ajouter et valider	`git add` puis `git commit -m "..."`	Cocher les fichiers, message, `Commit to ...`	`Git Commit -> ...` (cocher les fichiers)
Publier	`git push`	`Push origin`	`Push...`
Récupérer	`git fetch` / `git pull`	`Fetch origin` / `Pull origin`	`Fetch...` / `Pull...`
Créer une branche	`git switch -c nom`	`Current branch > New branch`	`Create Branch...`
Changer de branche	`git switch nom`	`Current branch`	`Switch/Checkout...`
Fusionner	`git merge nom`	`Branch > Merge into current branch`	`Merge...`
Résoudre un conflit	Éditer le fichier, `git add`, `git commit`	`Open in editor`, puis `Commit merge`	`Edit conflicts`, puis `Resolved`
Historique	`git log -oneline -graph -all`	Onglet `History`	`Show log`

Consigne : indiquez dans votre compte rendu l'outil utilisé par chaque membre de l'équipe. Les captures d'écran demandées peuvent provenir du terminal ou de l'interface graphique.

1.3. Utiliser Git Bash : lancement et authentification

Cette sous-section s'adresse aux étudiants qui travaillent sous Windows en ligne de commande.

Étape 1 : installer et lancer Git Bash

1. Installez Git for Windows (`https://git-scm.com/download/win`). Git Bash est installé avec lui ; conservez les options par défaut.
2. Lancez Git Bash par l'une de ces deux méthodes :
   — menu Démarrer, tapez `Git Bash`, puis ouvrez l'application ;
   — dans l'Explorateur de fichiers, faites un clic droit dans le dossier de travail puis choisissez `Open Git Bash here` (sous Windows 11 : `Afficher plus d'options` d'abord).
3. Vérifiez que Git fonctionne et placez-vous dans votre dossier de travail :

```
git --version
pwd
cd /c/Users/VotreNom/Documents
ls
```

Dans Git Bash, les chemins s'écrivent avec des barres obliques : le disque `C:` devient `/c/`. Pour coller du texte, utilisez `Shift+Insert` ou le clic droit (`Ctrl+V` ne fonctionne pas dans toutes les versions).

Étape 2 : s'authentifier auprès de GitHub

GitHub n'accepte plus le mot de passe du compte pour les opérations Git en HTTPS. Choisissez l'une des deux méthodes suivantes.

Méthode 1 : HTTPS avec connexion par navigateur.

1. Clonez le dépôt avec l'URL HTTPS (voir A4).
2. À la première opération qui demande une authentification (`clone` d'un dépôt privé, `push`), une fenêtre de Git Credential Manager (installé avec Git for Windows) s'ouvre : choisissez `Sign in with your browser`.
3. Connectez-vous à GitHub dans le navigateur et autorisez l'accès.
4. Les identifiants sont mémorisés : les opérations suivantes ne redemandent rien. Vérifiez avec `git push` ou `git pull`.

Si la fenêtre ne s'affiche pas, utilisez un *Personal Access Token* (GitHub : `Settings > Developer settings > Personal access tokens`) à la place du mot de passe. Ne l'enregistrez jamais dans un fichier du dépôt.

Méthode 2 : SSH (voir la Partie E).

1. Générez la clé, ajoutez la clé publique à GitHub et clonez ou configurez le dépôt avec une URL SSH (`git@github.com:...`).
2. Démarrez l'agent SSH et chargez votre clé. La phrase secrète est demandée une seule fois ; elle ne s'affiche pas pendant la saisie :

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

3. Testez la connexion. À la toute première connexion, SSH demande de confirmer l'empreinte du serveur : comparez-la à celle publiée dans la documentation GitHub, puis répondez `yes` :

```
ssh -T git@github.com
```

   Un message de bienvenue contenant votre nom d'utilisateur confirme que l'authentification fonctionne.

Une fois authentifié, vous pouvez exécuter toutes les commandes du TP (`clone`, `push`, `pull`, etc.) dans cette même fenêtre Git Bash. Si l'agent SSH n'est pas actif après la fermeture de Git Bash, relancez les deux commandes de l'étape 2.

2. Partie A – Création du dépôt et configuration de Git

A1. L'étudiant A crée un dépôt GitHub nommé `TP-Git-Collaboration` et invite les deux autres étudiants comme collaborateurs (`Settings > Collaborators`). Les étudiants B et C acceptent l'invitation reçue par e-mail ou dans leurs notifications GitHub.

A2. Chaque étudiant vérifie que Git est installé :

```
git --version
```

Mode graphique : installez GitHub Desktop ou TortoiseGit (ce dernier nécessite Git for Windows).

A3. Configurez votre identité Git avec votre nom et l'adresse e-mail associée à vos commits :

```
git config --global user.name "Prenom Nom"
git config --global user.email "votre-email@example.com"
git config --global --list
```

Mode graphique : dans GitHub Desktop, `File > Options > Git` ; dans TortoiseGit, `Settings > Git > Config`.

A4. Clonez le dépôt. Remplacez `UTILISATEUR` par le compte GitHub de l'étudiant A, qui héberge le dépôt :

```
git clone https://github.com/UTILISATEUR/TP-Git-Collaboration.git
cd TP-Git-Collaboration
git status
```

Mode graphique : utilisez `Clone repository` (GitHub Desktop) ou `Git Clone...` (TortoiseGit) avec la même URL.

A5. Créez les fichiers du projet et ajoutez un premier contenu dans `README.md`. Enregistrez cette première version :

```
git add README.md main.c module1.c module2.c module3.c .gitignore
git commit -m "Initialisation du projet"
git push origin main
```

Mode graphique : cochez les fichiers, saisissez le message « Initialisation du projet », validez, puis publiez (`Push`).

A6. Vérifiez l'historique avec `git log -oneline` (ou l'onglet `History` / `Show log`).

Question A : Quelle est la différence entre un dépôt local et un dépôt distant ?

3. Partie B – Développement collaboratif avec les branches

Chaque étudiant doit créer une branche dédiée à son module. Avant de commencer, synchronisez la branche principale.

```
git switch main
git pull origin main
git switch -c feature/module1
```

L'étudiant B utilise `feature/module2` et l'étudiant C utilise `feature/module3`.

Mode graphique : sélectionnez `main`, faites `Pull`, puis créez la branche (`New branch` ou `Create Branch...`) avec le même nom.

B1. Modifiez le fichier associé à votre rôle.

B2. Consultez les changements :

```
git status
git diff
```

B3. Ajoutez le fichier modifié et créez un commit descriptif :

```
git add module1.c
git commit -m "Ajout du module de saisie"
```

B4. Publiez votre branche :

```
git push -u origin feature/module1
```

B5. Sur GitHub, ouvrez une Pull Request de votre branche vers `main`. Présentez les changements et demandez une revue à un autre membre de l'équipe.

Mode graphique : GitHub Desktop propose le bouton `Create Pull Request` après la publication de la branche ; avec TortoiseGit, créez la Pull Request depuis le site GitHub.

Questions B :

- Quel est l'intérêt d'une branche par fonctionnalité ?
- À quoi sert un commit ?
- Quelle est la différence entre `push` et `pull` ?
- Quel est le rôle d'une Pull Request ?

4. Partie C – Synchronisation et notifications

C1. Après l'intégration d'une modification par un camarade, récupérez la version distante :

```
git switch main
git pull origin main
```

C2. Vérifiez les commits reçus :

```
git log --oneline --graph --all
```

C3. Sur GitHub, ouvrez la page du dépôt, sélectionnez `Watch` puis `Custom`, et configurez les notifications souhaitées.

C4. Vérifiez les paramètres de notification de votre compte GitHub et l'adresse e-mail associée. Envoyez un commit de test et vérifiez les notifications effectivement reçues.

À retenir : les notifications dépendent des préférences de l'utilisateur, du type d'événement et des paramètres du dépôt. Vérifiez leur fonctionnement plutôt que de supposer que chaque push déclenche systématiquement un e-mail.

5. Partie D – Provoquer et résoudre un conflit de fusion

5.1. D.1. Préparer la modification concurrente

L'étudiant C s'assure que `main.c` contient une ligne commune, par exemple :

```
printf("Application de securite informatique\n");
```

Les étudiants A et B créent chacun une branche à partir de la même version de `main`. Ils remplacent cette ligne par des messages différents.

Version de A :

```
printf("Application developpee par A\n");
```

Version de B :

```
printf("Application developpee par B\n");
```

Chaque étudiant enregistre et publie son changement :

```
git add main.c
git commit -m "Modification du message principal"
git push -u origin feature/conflit-A
```

L'étudiant B utilise sa propre branche, par exemple `feature/conflit-B`. Les deux branches doivent partir du même état initial. L'étudiant C fusionne d'abord la Pull Request de la branche A dans `main` (sur GitHub, bouton `Merge pull request`).

5.2. D.2. Fusionner la branche B

L'étudiant B synchronise `main`, puis tente d'intégrer ses changements dans une branche de travail à jour :

```
git fetch origin
git switch feature/conflit-B
git merge origin/main
```

Mode graphique : faites `Fetch origin`, placez-vous sur `feature/conflit-B`, puis fusionnez `origin/main` (`Merge into current branch` dans GitHub Desktop, `Merge...` dans TortoiseGit).

Si les mêmes lignes ont été modifiées différemment, Git signale un conflit. Le fichier peut contenir des marqueurs de ce type :

```
<<<<<<< HEAD
printf("Application developpee par B\n");
=======
printf("Application developpee par A\n");
>>>>>>> origin/main
```

Le contenu exact dépend du sens de la fusion : dans cet exemple, `HEAD` correspond à la branche courante, et l'autre section correspond à la branche fusionnée.

5.3. D.3. Résoudre le conflit

D1. Ouvrez `main.c` et choisissez le résultat souhaité. Vous pouvez retenir un seul message ou combiner les informations.

Mode graphique : GitHub Desktop ouvre le fichier dans votre éditeur (`Open in ...`) ; TortoiseGit propose `Edit conflicts`, qui ouvre un outil de fusion à trois volets.

D2. Supprimez les marqueurs `«««<`, `=======` et `»»»>`.

D3. Vérifiez le fichier puis compilez et testez l'application.

D4. Marquez le conflit comme résolu et créez le commit de fusion :

```
git status
git add main.c
git commit -m "Resolution du conflit dans main.c"
```

Mode graphique : dans TortoiseGit, cliquez sur `Resolved` puis `Commit` ; dans GitHub Desktop, une fois le conflit supprimé, cliquez sur `Commit merge`.

D5. Publiez la branche et proposez une Pull Request vers `main` :

```
git push origin feature/conflit-B
```

Questions D :

- Pourquoi Git ne peut-il pas choisir automatiquement entre les deux versions ?
- Que représentent les marqueurs de conflit ?
- Quelle est la différence entre résoudre un conflit et annuler une modification ?
- Pourquoi faut-il tester le programme après une fusion ?

6. Partie E – Authentification SSH et clés cryptographiques

6.1. E.1. Générer une paire de clés

Chaque étudiant génère sa propre paire de clés sur sa machine. Sous Windows, ouvrez Git Bash ; sous Linux, utilisez un terminal. Cette étape se fait en ligne de commande quel que soit l'outil choisi pour le reste du TP.

```
ssh-keygen -t ed25519 -C "votre-email@example.com"
```

Acceptez le chemin proposé et définissez une phrase secrète. La clé privée reste sur votre machine et ne doit jamais être partagée.

6.2. E.2. Ajouter la clé publique à GitHub

Affichez uniquement la clé publique :

```
cat ~/.ssh/id_ed25519.pub
```

Dans GitHub, ouvrez `Settings > SSH and GPG keys > New SSH key`, puis ajoutez la clé publique. Ne copiez jamais le fichier `id_ed25519` sans l'extension `.pub` dans GitHub ou dans un dépôt.

Testez l'authentification :

```
ssh -T git@github.com
```

Si le dépôt a été cloné en HTTPS, configurez l'URL SSH du dépôt distant après avoir vérifié le nom du propriétaire :

```
git remote -v
git remote set-url origin git@github.com:UTILISATEUR/TP-Git-Collaboration.git
```

Mode graphique : dans TortoiseGit, ouvrez `Settings > Network` et choisissez comme client SSH l'exécutable `ssh.exe` de Git for Windows pour réutiliser la clé générée ci-dessus (ou utilisez PuTTYgen et Pageant, avec une clé au format PuTTY). Pour changer l'URL du dépôt distant : `Settings > Git > Remote`. Dans GitHub Desktop, `Repository > Repository settings > Remote`. Le comportement de GitHub Desktop avec SSH dépend de la configuration de la machine ; en cas de difficulté, effectuez le test `ssh -T git@github.com` dans un terminal.

6.3. E.3. Questions de sécurité

E1. Expliquez la différence entre une clé publique et une clé privée.

E2. Pourquoi chaque étudiant doit-il disposer de sa propre paire de clés ?

E3. Quel est le rôle de la phrase secrète protégeant la clé privée ?

E4. Quelle est la différence entre l'authentification SSH et la signature cryptographique d'un commit ?

E5. Pourquoi ne faut-il jamais publier une clé privée dans un dépôt Git ?

Documentation officielle :

- Git : https://git-scm.com/docs
- GitHub et les clés SSH : https://docs.github.com/en/authentication/connecting-to-github-with-ssh
- Notifications GitHub : https://docs.github.com/en/account-and-profile/managing-subscriptions-an
- GitHub Desktop : https://docs.github.com/en/desktop
- TortoiseGit : https://tortoisegit.org/docs/

7. Partie F – Ignorer certains fichiers avec `.gitignore`

Dans un projet logiciel, certains fichiers ne doivent généralement pas être versionnés : fichiers temporaires, fichiers compilés, paramètres locaux ou secrets. Git utilise le fichier `.gitignore` pour ignorer les fichiers non suivis qui correspondent aux règles définies.

7.1. F.1. Créer et configurer `.gitignore`

À la racine du dépôt, créez un fichier nommé `.gitignore` contenant, par exemple :

```
# Fichiers compiles en C
*.o
*.exe
programme

# Fichiers temporaires et journaux
*.tmp
*.log

# Parametres locaux et secrets
.env
config.local

# Dossiers d'outils et editeurs
.vscode/
.idea/
```

```
# Systeme d'exploitation
.DS_Store
Thumbs.db
```

Les motifs sont interprétés par Git. Par exemple, `*.o` ignore les fichiers dont le nom se termine par `.o`, tandis que `.vscode/` ignore ce dossier.

7.2. F.2. Vérifier qu'un fichier est ignoré

F1. Créez un fichier temporaire, par exemple `test.tmp`, puis vérifiez l'état du dépôt :

```
git status
```

F2. Le fichier `test.tmp` ne doit pas apparaître parmi les fichiers non suivis.

F3. Pour connaître la règle qui ignore un fichier, utilisez :

```
git check-ignore -v test.tmp
```

Mode graphique : TortoiseGit permet aussi d'ajouter un fichier aux règles par clic droit (`TortoiseGit > Add to ignore list`). GitHub Desktop propose `Ignore file` par clic droit sur un fichier de l'onglet `Changes`. La commande `git check-ignore -v` s'exécute dans un terminal.

F4. Ajoutez et versionnez le fichier `.gitignore` :

```
git add .gitignore
git commit -m "Ajout des regles .gitignore"
git push
```

7.3. F.3. Empêcher l'ajout de fichiers au commit

Avant chaque commit, examinez les changements :

```
git status
git diff
git diff --staged
```

Si un fichier est déjà suivi par Git, l'ajouter à `.gitignore` ne suffit pas à cesser de le versionner. Pour le retirer de l'index tout en le conservant sur la machine :

```
git rm --cached config.local
git commit -m "Retrait du fichier local du suivi Git"
```

Mode graphique : dans TortoiseGit, utilisez `Delete (keep local)` dans le menu clic droit, puis validez. GitHub Desktop n'offre pas d'équivalent direct : utilisez le terminal pour cette commande.

Vérifiez que le fichier est bien couvert par une règle de `.gitignore` avant de poursuivre. La commande `git rm -cached` retire le fichier de l'index ; elle ne l'efface pas du répertoire de travail.

7.4. F.4. Exercice pratique

F1. Ajoutez au dépôt un fichier `notes.tmp` et vérifiez qu'il est ignoré.

F2. Créez un fichier `config.local` et ajoutez-le aux règles de `.gitignore`.

F3. Utilisez `git check-ignore -v config.local` pour identifier la règle correspondante.

F4. Vérifiez avec `git status` que ces fichiers ne sont pas proposés pour le commit.

F5. Ajoutez `.gitignore` au commit et poussez les changements.

F6. Si un fichier sensible a déjà été publié, expliquez pourquoi l'ajouter à `.gitignore` ne suffit pas à supprimer son historique. Ne placez jamais de vrais mots de passe, jetons ou clés privées dans le dépôt.

Questions F :

— Quelle est la différence entre un fichier ignoré et un fichier déjà suivi par Git ?
— À quoi sert la commande `git check-ignore -v` ?
— Pourquoi faut-il vérifier `git status` avant un commit ?
— Pourquoi un fichier `.env` contenant des secrets ne doit-il pas être publié ?

8. Partie G – Partie avancée : reset, fork, branches, conflits et fusions

Cette partie approfondit les notions vues plus haut. Réalisez les manipulations dans un dépôt de test (ou sur une branche dédiée comme `feature/avance`) afin de ne pas abîmer l'historique du projet principal.

8.1. G.1. Annuler et réinitialiser des commits

Annuler des modifications non commitées

```
git restore fichier.c            # abandonne les modifications du fichier
git restore --staged fichier.c   # retire le fichier de l'index (git add annule)
```

Corriger le dernier commit

```
git commit --amend -m "Nouveau message"
```

Reset : déplacer la branche vers un commit antérieur

```
git log --oneline
git reset --soft HEAD~1    # annule le commit, garde les changements dans l'index
git reset --mixed HEAD~1   # annule le commit, garde les changements non indexes (defaut)
git reset --hard HEAD~1    # annule le commit ET supprime les changements
git reset --hard <hash>    # revient a un commit precis
```

Option	Historique	Index	Fichiers de travail
`-soft`	modifié	conservé	conservés
`-mixed`	modifié	réinitialisé	conservés
`-hard`	modifié	réinitialisé	**perdus**

Revert : annuler un commit en conservant l'historique

```
git revert <hash>
```

`git revert` crée un nouveau commit qui inverse les changements d'un commit précédent. C'est la méthode à utiliser pour un commit déjà publié.

Retrouver un état perdu

```
git reflog
git reset --hard HEAD@{2}
```

`git reflog` liste les derniers déplacements de `HEAD`, y compris après un reset, ce qui permet souvent de récupérer des commits « perdus ».

Attention :

- `git reset -hard` est destructif : les modifications non commitées sont perdues. Vérifiez avec `git status` avant de l'utiliser.
- Ne réécrivez pas l'historique de commits déjà poussés sur une branche partagée (`main`) : utilisez `git revert`.
- Sur votre propre branche de fonctionnalité, après un reset ou un amend d'un commit déjà poussé, la publication exige `git push -force-with-lease`, plus sûr que `-force`. Prévenez vos coéquipiers avant de le faire.

Mode graphique : dans GitHub Desktop, onglet `History`, clic droit sur un commit : `Revert Changes in Commit` ; le bouton `Undo` (sous la zone de message) annule le dernier commit non publié, et `Amend` permet de modifier le dernier commit. Dans TortoiseGit : `Show log`, clic droit sur un commit, `Reset "branche" to this...` (choix `Soft`, `Mixed` ou `Hard`) ou `Revert change by this commit`.

Exercice G.1

G1.1. Créez trois commits successifs modifiant `README.md`.

G1.2. Annulez le dernier avec `git reset -soft HEAD~1`, puis refaites le commit avec un meilleur message.

G1.3. Faites un `git reset -hard HEAD~1`, puis retrouvez le commit supprimé avec `git reflog`.

G1.4. Sur un commit déjà poussé, utilisez `git revert` et observez l'historique avec `git log -oneline`.

8.2. G.2. Le fork

Un *fork* est une copie personnelle, hébergée sur votre compte GitHub, d'un dépôt appartenant à quelqu'un d'autre. Il permet de proposer des modifications à un projet sans en être collaborateur. Le dépôt d'origine est appelé *upstream*, votre copie *origin*.

G2.1. Sur la page du dépôt d'origine, cliquez sur `Fork`, puis créez le fork sur votre compte.

G2.2. Clonez **votre fork** (et non le dépôt d'origine) :

```
git clone https://github.com/VOTRE-COMPTE/TP-Git-Collaboration.git
cd TP-Git-Collaboration
```

G2.3. Déclarez le dépôt d'origine comme second dépôt distant :

```
git remote add upstream https://github.com/UTILISATEUR/TP-Git-Collaboration.git
git remote -v
```

G2.4. Travaillez sur une branche, puis publiez-la sur votre fork :

```
git switch -c feature/amelioration-readme
git add README.md
git commit -m "Amelioration du README"
git push -u origin feature/amelioration-readme
```

G2.5. Sur GitHub, ouvrez une Pull Request de la branche de votre fork vers la branche `main` du dépôt d'origine.

G2.6. Tenez votre fork à jour avec le dépôt d'origine :

```
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

GitHub propose aussi le bouton `Sync fork` sur la page du fork ; il met à jour la branche du fork sur GitHub, après quoi il faut faire `git pull` en local.

Mode graphique : GitHub Desktop propose de créer un fork au moment du clonage d'un dépôt dont vous n'êtes pas collaborateur (option `To contribute to the parent project`). Dans TortoiseGit, ajoutez `upstream` dans `Settings > Git > Remote`, puis utilisez `Fetch...` et `Merge...`.

Exercice G.2

L'étudiant C joue le rôle de contributeur externe : même s'il est collaborateur, il crée un fork du dépôt de l'étudiant A, modifie le `README.md`, puis propose une Pull Request que l'étudiant A examine et fusionne.

8.3. G.3. Commandes de gestion des branches

```
git branch                          # liste les branches locales
git branch -a                       # locales et distantes
git branch -vv                      # dernier commit et branche suivie
git branch nom                      # cree une branche sans s'y placer
git switch nom                      # change de branche
git switch -c nom                   # cree et change de branche
git branch -m ancien nouveau        # renomme une branche
git branch --merged                 # branches deja fusionnees dans la courante
git branch -d nom                   # supprime une branche fusionnee
git branch -D nom                   # supprime de force (non fusionnee)
git push -u origin nom              # publie et associe la branche distante
git push origin --delete nom        # supprime la branche distante
git fetch --prune                   # retire les references distantes supprimees
```

Mode graphique : GitHub Desktop : menu `Current branch` (création, changement de branche) et menu `Branch` (renommer, supprimer). TortoiseGit : `Create Branch...`, `Switch/Checkout...` et `Browse references` (renommer, supprimer).

Exercice G.3

G3.1. Créez les branches `feature/test1` et `feature/test2` et listez-les avec `git branch -vv`.

G3.2. Renommez `feature/test2` en `feature/essai`.

G3.3. Publiez `feature/test1`, puis supprimez-la en local et sur le dépôt distant.

G3.4. Utilisez `git fetch -prune` et vérifiez avec `git branch -a`.

8.4. G.4. Gérer les conflits

Un conflit apparaît lorsque Git ne peut pas combiner automatiquement deux modifications des mêmes lignes (voir la Partie D). Pour les gérer méthodiquement :

```
git status                                  # fichiers "both modified"
git diff --name-only --diff-filter=U        # liste des fichiers en conflit
git merge --abort                           # abandonne la fusion et revient avant
```

Choisir une version complète d'un fichier

Pendant une fusion, « ours » désigne la branche courante et « theirs » la branche fusionnée :

```
git checkout --ours main.c      # garde la version de la branche courante
git checkout --theirs main.c    # garde la version de la branche fusionnee
git add main.c
git commit
```

Afficher aussi l'ancêtre commun et utiliser un outil de fusion

```
git config --global merge.conflictstyle diff3
git mergetool
```

Avec `diff3`, les marqueurs contiennent une section supplémentaire (`|||||||`) qui montre la version d'origine, ce qui aide à comprendre qui a modifié quoi.

Conflits pendant un rebase

```
git rebase main
# resolution du conflit dans les fichiers, puis :
git add fichier.c
git rebase --continue
git rebase --abort      # pour abandonner
```

Pendant un rebase, les rôles de « ours » et « theirs » sont inversés par rapport à un merge.

Limiter les conflits

Faites de petits commits, synchronisez souvent avec `git pull`, répartissez les fichiers entre les membres de l'équipe et communiquez avant de modifier un fichier partagé.

Exercice G.4

G4.1. Créez deux branches modifiant différemment `module1.c` *et* `main.c`, puis fusionnez-les afin d'obtenir deux fichiers en conflit.

G4.2. Utilisez `git diff -name-only -diff-filter=U` pour les lister.

G4.3. Résolvez `main.c` en gardant la version de la branche courante (`-ours`) et `module1.c` en combinant manuellement les deux versions.

G4.4. Terminez la fusion, compilez et testez. Recommencez ensuite le scénario et utilisez `git merge -abort`.

8.5. G.5. Fusionner des branches

Placez-vous sur la branche qui doit recevoir les changements, puis fusionnez l'autre :

```
git switch main
git merge feature/module1
```

Stratégie	Principe	Historique obtenu
Fast-forward	`main` n'a pas avancé : Git déplace simplement le pointeur	linéaire, sans commit de fusion
`-no-ff`	force un commit de fusion même si le fast-forward est possible	historique en « bulles » montrant la branche
`-squash`	regroupe tous les commits de la branche en un seul changement à valider	un seul commit, sans lien avec la branche
`rebase`	rejoue les commits de la branche au-dessus de la cible	linéaire, commits réécrits

```
git merge --no-ff feature/module2
git merge --squash feature/module3
git commit -m "Ajout du module d'affichage"

git switch feature/module1
git rebase main
```

Pour visualiser le résultat :

```
git log --oneline --graph --all --decorate
```

Sur GitHub, une Pull Request peut être fusionnée de trois façons : `Create a merge commit`, `Squash and merge` ou `Rebase and merge`. La règle d'or : ne faites pas de `rebase` sur une branche déjà partagée avec d'autres personnes.

Mode graphique : GitHub Desktop : `Branch > Merge into current branch...` ; `Branch > Rebase current branch...` est également disponible. TortoiseGit : `Merge...` (avec les options `No Fast Forward` et `Squash`) et `Rebase...`.

Exercice G.5

G5.1. Créez `feature/ff` à partir de `main`, ajoutez un commit et fusionnez-la : le fast-forward doit se produire.

G5.2. Créez deux branches qui divergent de `main` (sans modifier les mêmes lignes), fusionnez la première puis la seconde avec `-no-ff`.

G5.3. Refaites un scénario similaire avec `-squash`, puis comparez les trois historiques avec `git log -oneline -graph -all`.

G5.4. Faites un `rebase` d'une branche locale non publiée sur `main` et expliquez la différence avec un `merge`.

Questions G :

- Quelle est la différence entre `reset -soft`, `-mixed` et `-hard` ?
- Pourquoi préférer `git revert` à `git reset` pour un commit déjà publié ?
- Quelle est la différence entre un fork et un clone ? Et entre un fork et une branche ?
- À quoi sert le dépôt distant `upstream` ?
- Que se passe-t-il lorsqu'on supprime une branche locale : la branche distante est-elle supprimée aussi ?
- Quand un fast-forward est-il possible ? Pourquoi utiliser `-no-ff` ?
- Quels sont les avantages et les risques du `rebase` par rapport au `merge` ?

9. Livrables à remettre

Chaque équipe remet :

1. l'URL du dépôt GitHub ;
2. les trois modules sources et le programme principal ;
3. l'historique des commits et les branches créées ;
4. une capture d'écran du conflit et de sa résolution (terminal ou interface graphique) ;
5. une preuve de configuration SSH, sans révéler de clé privée ;
6. les preuves des exercices de la partie avancée (historiques `git log -graph`, captures du fork et de la Pull Request) ;
7. un court compte rendu répondant aux questions du TP et précisant l'outil utilisé par chaque membre (ligne de commande, GitHub Desktop, TortoiseGit, etc.).

10. Barème indicatif

Critère	Points
Création du dépôt et configuration Git	2
Branches, commits, push et pull	3
Conflit de fusion résolu et programme testé	3
Notifications et collaboration	1
Authentification SSH et compréhension des clés	3
Livrables et qualité du compte rendu	2
Total	**14/14**

Fin du TP – Travaillez avec des commits clairs, synchronisez régulièrement vos branches et ne partagez jamais vos clés privées.